AWS IoT Cost Review

Realtek IoT Cloud Platform — Cost per Device Evaluation Feedback

2026-06-30

This document contains our review feedback, organized into four parts:

1. **Scenario Confirmation** — confirming our understanding of your inputs
2. **Top 5 Cost Items Analysis** — findings and questions per cost driver
3. **Additional Questions** — other items needed to finalize the estimate
4. **EKS Load Test** — sizing review methodology recommendation

Part 1: Scenario Confirmation

Based on your documents, we understand the target scenario as follows. Please correct us if any assumption is wrong.

Parameter	Value	Source
Registered IoT Devices	100,000 (always-online MQTT)	iot_cloud_status_report P2, P4
End Users	5,000	iot_cloud_status_report P4
Devices per User	20	iot_cloud_status_report P4
Region (cost estimate)	ap-southeast-1	iot_cloud_status_report P5
Region (load test)	us-east-1	Email 6/24
Telemetry frequency	12 msg/hr/device (~every 5 min)	iot_cloud_status_report P5
Message size	≤1 KB	iot_cloud_status_report P5
Shadow update frequency	1/hr/device	iot_cloud_status_report P5
Downlink frequency	1/day/device	iot_cloud_status_report P5
Database	PostgreSQL, ~1 TB storage	iot_cloud_status_report P5
Cost allocation	10% user pool / 90% device pool	iot_cloud_status_report P4
Video/WebRTC/TURN	Excluded from this estimate	iot_cloud_status_report P2
Scope	Connectivity + device mgmt only	Meeting 6/23

Part 2: Top 5 Cost Items — Analysis & Questions

Per your suggestion to focus on the top 5 cost drivers, we have organized our analysis accordingly:

#	Cost Item	Realtek Estimation	Our Analysis
1	CloudHSM	$2,336/month	Alternative analysis + questions
2	AWS IoT Core (connectivity + messaging)	$1,650/month	Rules Engine gap + Basic Ingest opportunity
3	IoT Device Management	$1,135/month	Managed Integrations not applicable
4	RDS PostgreSQL	$557/month	Sizing & throughput questions
5	ECS Fargate + Lambda	$646/month	Workload division clarification needed

#1: CloudHSM ($2,336/month per instance)

Our assumption of your use case:

Based on your aws-iot-flow.pdf (Page 2, Device Activation Flow), CloudHSM appears in the path: Device SDK → Lambda → IoT Core registry → **CloudHSM/KMS** → RDS. We assume the role of CloudHSM here is:

- Protect the CA private key used to sign X.509 device certificates during device onboarding / factory enrollment
- Corresponding to the "Cert Issuer" service in your current K8s architecture (backed by OpenBao)

Please confirm if this understanding is correct, or if CloudHSM serves additional purposes in your architecture.

When is CloudHSM actually required?

CloudHSM is designed for scenarios that require:

- PKCS#11 direct access or custom key ceremony (e.g., banking-grade key management)
- Regulatory mandates requiring a dedicated, single-tenant HSM instance (e.g., PCI-DSS in some interpretations)
- Crypto algorithms not yet supported by KMS (e.g., custom/legacy algorithms)

If your requirement is specifically "protect a CA private key for device cert signing," there are lighter-weight alternatives available within the AWS IoT ecosystem.

Alternatives for device certificate signing — feature comparison: (Option A/B/C/D/E)

Feature	A. IoT Core Built-in	B. BYOCA + KMS	C. ACM Private CA	D. CloudHSM
Monthly cost	$0	~$8	~$433*	$2,336+
Signing interface	IoT Core MQTT API (CreateKeysAndCertificate/ CreateCertificateFromCsr)	KMS Sign API	ACM PCA IssueCertificate API	PKCS#11 (C_Sign)
Additional implementation required	None — fully managed by IoT Core (zero code)	Lambda (cert assembly + CSR validation) + DynamoDB (serial number tracking) + X.509 builder code (e.g., Python cryptography lib)	Lambda (only if using Fleet Provisioning — single API call to IssueCertificate, no cert assembly needed). JITP mode = zero code.	PKCS#11 client integration + cert assembly code (same as B) + HSM cluster management (cross-AZ, backup, user mgmt)
PQC (ML-DSA)	✗ RSA-2048 only	✓ ML_DSA_44/65/87	✓ ML_DSA_44/65/87	✗ Not yet supported
FIPS 140-3 Level 3	✓ (AWS internal HSM)	✓ Validated	✓ (underlying HSM)	✓ Validated
Signing algorithms	RSA-2048	RSA/ECC/ML-DSA	RSA/ECC/ML-DSA	RSA/ECC (no PQC)
CA identity	AWS (Amazon Root CA)	Realtek (key stays in KMS)	Realtek (key stays in ACM PCA)	Realtek (key exportable)
Issued cert portability	✗ Tied to AWS IoT	✓ Portable	✓ Portable	✓ Portable
CA key portability	N/A (AWS owns CA)	✗ Non-extractable	✗ Non-extractable	✓ Exportable (C_WrapKey)
Key ceremony	✗ Not visible	✗ CloudTrail audit only	✗ CloudTrail audit only	✓ Full ceremony

References:

- **[A] IoT Core CreateKeysAndCertificate API:** docs.aws.amazon.com/iot/latest/developerguide/provision-wo-cert.html
- **[B] KMS Sign API reference:** docs.aws.amazon.com/kms/latest/APIReference/API_Sign.html
- **[C] ACM PCA IssueCertificate API:** docs.aws.amazon.com/privateca/latest/APIReference/API_IssueCertificate.html
- **[D] CloudHSM PKCS#11 SDK (C_Sign):** docs.aws.amazon.com/cloudhsm/latest/userguide/pkcs11-mechanisms.html

Additional references:

- **KMS ML-DSA (PQC):** docs.aws.amazon.com/kms/latest/developerguide/mldsa.html
- **KMS FIPS 140-3 Level 3:** aws.amazon.com/blogs/security/aws-kms-now-fips-140-2-level-3-what-does-this-mean-for-you/
- **CloudHSM PKCS#11 mechanisms:** docs.aws.amazon.com/cloudhsm/latest/userguide/pkcs11-v3-mechanisms.html
- **ACM Private CA algorithms (incl. ML-DSA):** docs.aws.amazon.com/privateca/latest/userguide/PcaWelcome.html

Initial assessment: For fully managed with maximum compliance: **Option A (IoT Core Built-in)** if AWS-owned CA is acceptable. If the CA must be Realtek-owned: **Option C (ACM Private CA)** for fully managed operation. If you additionally require a hardware Root CA with FIPS Level 3 key ceremony for the root of trust: **Option E (Hybrid: CloudHSM offline Root + ACM PCA operational)** described below.

Option E: Hybrid PKI — CloudHSM (offline Root CA) + ACM Private CA (operational signing)

This architecture separates the Root CA (high-security, rarely used) from daily certificate signing (high-volume, fully managed):

Phase 1 — Root CA Setup (CloudHSM, temporary): Create HSM cluster → generate Root CA key pair inside HSM → sign Intermediate CA certificate → confirm backup → delete HSM instance (stop billing).

Phase 2 — Intermediate CA Activation: Create Subordinate CA in ACM Private CA → import Intermediate CA certificate signed by Root → ACM PCA begins serving device certificate issuance.

Phase 3 — Steady-State: ACM PCA handles all device/server cert issuance automatically. No CloudHSM running.

Phase 4 — Root CA Maintenance (occasional): Re-activate CloudHSM only when needed (new Intermediate CA, renewal, CRL update, disaster recovery). Flow: restore HSM from backup → sign → confirm backup → delete HSM. Typically a few hours per year.

Ongoing cost:

• ACM PCA: $400/month per CA (fixed)

• Cert issuance: $0.75/cert (1-1K), $0.35 (1K-10K), $0.001 (10K+). At 100K devices: Tiered: first 1K × $0.75 + next 9K × $0.35 + remaining 90K × $0.001 = ~$3,990 one-time (best case: issue all in one calendar month). Amortized over 10 years: ~$33/month..

• CloudHSM: ~$3.20/hr, only online a few hours/year → ~$10-20/year (negligible)

• CloudHSM backup storage: Free

Bottom line: Primary cost = ACM PCA $400/month + per-cert fees. CloudHSM cost is negligible for offline Root CA use case. Total: ~$400/month vs $2,336/month for always-on CloudHSM — saving $1,936/month while achieving higher security (hardware Root CA + managed operational CA).

Ref:

CloudHSM Pricing — https://aws.amazon.com/cloudhsm/pricing/

ACM PCA Pricing — https://aws.amazon.com/private-ca/pricing/

Cognito Pricing — https://aws.amazon.com/cognito/pricing/

All options at a glance:

	A. IoT Core	B. BYOCA+KMS	C. ACM PCA	D. CloudHSM	E. Hybrid
Monthly cost	$0	~$8	~$433*	$2,336+	~$433*
CA identity	AWS	Realtek	Realtek	Realtek	Realtek
Root key security	AWS internal	KMS (FIPS L3)	PCA internal	HSM (FIPS L3)	HSM offline (FIPS L3)
PQC-ready	✗	✓	✓	✗	✓ (via PCA)
Implementation	Zero	High	Low	High	Medium (one-time)
HA included	✓	✓	✓	Need 2nd HSM	✓ (PCA handles)
Best for	Simplest, no custom CA needed	Lowest cost + own CA	Fully managed + own CA	Regulatory mandate for dedicated HSM	Highest security + cost controlled
Cert issuance fee (100K devices, one-time)	$0	~$1.50 (negligible)	~$3,990 (~$33/mo amortized over 10yr)	$0	~$3,990 (~$33/mo amortized over 10yr)

Questions to confirm:

1. Is there a specific regulatory or compliance requirement that mandates a dedicated HSM instance (PKCS#11, key ceremony)?
2. What signing algorithm does your Cert Issuer currently use? (RSA-2048, ECDSA P-256, or planning for PQC?)
3. Do you require the certificate Issuer to be Realtek's own CA? Or is it acceptable for device certificates to be signed by AWS IoT Core's built-in CA (Amazon Root CA)?

Potential savings vs CloudHSM ($2,336/month):

Option	Monthly cost	Savings vs CloudHSM
A. IoT Core Built-in	$0	−$2,336 (−100%)
B. BYOCA + KMS	~$8	−$2,328 (−99.7%)
C. ACM Private CA	~$433*	−$1,903 (−83%)
D. CloudHSM (always-on)	$2,336	baseline
E. Hybrid (CloudHSM offline + PCA)	~$433*	−$1,903 (−83%)

**** Options C/E monthly cost includes cert issuance amortized over 10 years: ~$3,990 (Sign 100K pcs in one month) ÷ 120 months ≈ $33/month. Options A/B/D have no per-cert signing fee from ACM PCA.***

#2: AWS IoT Core ($1,650/month)

Realtek estimation: Connectivity $415 + Messaging $1,037 (864M messages) + Shadow $86 + Shadow ops $108 = ~$1,650 (iot_cloud_status_report P5)

Finding: Rules Engine cost may be missing

In aws-iot-flow.pdf Page 3, telemetry flows through IoT Rules to Lambda/S3. If so, there is an additional charge:

- Rules triggered: 864M × $0.18/M = $155/month
- Actions executed: 864M × $0.18/M = $155/month
- **Total missing: ~$311/month** *(Rate $0.18/M estimated from ap-southeast-1 regional pattern — other IoT Core rates show consistent +20% vs eu-west-1.)*

Optimization: Basic Ingest

If telemetry is only consumed by Rules Engine (not subscribed by App clients), devices can publish to **$aws/rules/<rule-name>/topic**. This bypasses the message broker, eliminating the $1,037 messaging fee.

- **Net effect: Messaging $1,037 removed, Rules $311 added → save ~$726/month**

Questions:

1. **Telemetry data path:** (A) Rules → Lambda → RDS, (B) Rules → S3 directly, (C) Other?
2. **Does any App client subscribe to device telemetry topics via MQTT?** (If yes, Basic Ingest cannot be used)
3. **Does your telemetry data topic specifically require retained messages or App-side MQTT subscription?** (Basic Ingest applies per-topic — other topics in the same device connection can still use regular broker with retained messages.)

Note: Connectivity minutes uses 30 days/month = 43,200 min. AWS Calculator uses 30.4 days = 43,800 min. Difference is ~$6/month (negligible).

Ref:

IoT Core Pricing — https://aws.amazon.com/iot-core/pricing/

Basic Ingest — https://docs.aws.amazon.com/iot/latest/developerguide/iot-basic-ingest.html

IoT Core Quotas — https://docs.aws.amazon.com/general/latest/gr/iot-core.html

#3: IoT Device Management ($1,135/month)

Realtek estimation: 100K devices × $0.01/device/month = $1,000 (Managed Integrations) + $135 (Fleet Indexing)

Finding: Managed Integrations (MI) is not applicable

The $0.01/device/month pricing is for Managed Integrations (MI), a distinct feature designed for a specific ecosystem scenario:

- MI is a managed platform for Smart Home hub/gateway manufacturers to unify control of devices from multiple brands (ZigBee, Z-Wave, Wi-Fi) through a single API, using 80+ pre-defined Matter-based data model templates.
- It provides built-in Cloud-to-Cloud (C2C) connectors for integrating with third-party ecosystems (e.g., Alexa, Google Home), removing the need for custom integrations.
- Essentially, MI targets manufacturers who want to join the AWS-managed Smart Home ecosystem — AWS handles MQTT topics, IoT policies, rules, and message routing on their behalf. The $0.01/device fee covers this full managed abstraction layer.

Why this does not apply to Realtek:

- Realtek devices connect directly to MQTT via your own firmware SDK — not routing through a hub/gateway, not using Matter data models or C2C connectors.
- You are building your own device management logic (registry, shadow, OTA) on top of IoT Core primitives — this is the standard IoT Core + Fleet Indexing usage model.
- **Region availability:** MI is currently only available in Canada (Central) and Europe (Ireland). It is **not available in ap-southeast-1**, your target deployment region.

Revised estimate:

Item	Original	Revised
Managed Integrations subscription	$1,000/month	$0 (not applicable)
Fleet Indexing updates (60M × $2.25/M)	$135/month	$135/month
Total IoT Device Management	**$1,135/month**	**$135/month**

Impact: −$1,000/month

Ref:

Managed Integrations — https://docs.aws.amazon.com/iot-mi/latest/devguide/what-is-managedintegrations.html

IoT DM Pricing — https://aws.amazon.com/iot-device-management/pricing/

#4: RDS PostgreSQL ($557/month)

Realtek estimation: Single-AZ db.r7g.xlarge (4 vCPU, 32 GiB), 1 TB gp3 storage, 730 hrs/month

Questions:

1. **Is RDS the only database?** Your three data flows (User Login, Device Activation, MQTT Runtime) all point to RDS. Does user profiles, device registry, AND telemetry data all write to the same instance?
2. **Write throughput:** If 864M telemetry messages/month each trigger a write, that is ~333 writes/sec sustained. Can a single db.r7g.xlarge handle this alongside user/device queries?
3. **1 TB storage composition:** What data is stored? What is the retention policy? (Telemetry at 864M msg/month × 1KB = ~800 GB/year if not purged)

Architecture context: Based on your three data flows, we observe that RDS may be serving three distinct workloads with very different access patterns:

(a) User profile (5K users, low-frequency read/write) — if this is key-value lookup by user_id, Cognito User Pool attributes or DynamoDB may be more cost-effective than RDS.

(b) Device registry (100K devices, write-once at activation) — IoT Core Thing Attributes + Fleet Indexing ($135/month, already in estimate) provides device search without RDS.

(c) Telemetry (864M msg/month) — if this is append-only time-series data not requiring real-time SQL query, IoT Core Rule → Kinesis Data Firehose → S3 (Parquet) is significantly more cost-effective (~$51/month vs Lambda+RDS ~$3,500/month), with Athena for ad-hoc analysis. Device Shadow handles real-time "latest state" queries.

Alternative consideration — Aurora PostgreSQL:

	Aurora Standard (est.)	**RDS Multi-AZ (Basic HA)**
Storage durability	6 copies, 3 AZs (built-in)	2 copies, 2 AZs (EBS sync)
Failover	<30 seconds	1-2 minutes
Auto-scale storage	✓ Pay only for used data	✗ Fixed allocation (both instances)
Read replica add	✓ Instant (shared volume)	Additional instance + full copy
Est. monthly (200GB actual)	~$572	~$957
Savings vs RDS Multi-AZ	**−$385/month (−40%)**	baseline

Your current estimate uses RDS Single-AZ ($557/month). Aurora PostgreSQL offers significantly higher durability at minimal cost increase: storage layer automatically maintains 6 copies across 3 AZs (vs Single-AZ = 1 copy), failover in <30 seconds, and auto-scaling storage. Estimated cost: ~$620/month (+$63, +11%). For comparison, RDS Multi-AZ (equivalent HA level) would cost ~$957/month. Aurora delivers better protection at lower cost than RDS Multi-AZ.

Actual data stored	**RDS gp3 (1024 GB allocated)**	**Aurora I/O-Opt (pay for used)**	**Delta**
200 GB	$560 (inst $419 + 1024GB×$0.138)	$681 (inst $631 + 200GB×$0.248)	+$121
500 GB	$560 (same — within allocation)	$755 (inst $631 + 500GB×$0.248)	+$195
800 GB	$560 (nearing 80% capacity)	$830 (inst $631 + 800GB×$0.248)	+$270
1,000 GB	$560 (at capacity limit)	$879 (inst $631 + 1000GB×$0.248)	+$319
HA level included	✗ Single-AZ, no replication	✓ 6 copies / 3 AZ, <30s failover	—

Key takeaway: Aurora I/O-Optimized costs more than RDS Single-AZ (+$121 at 200GB), but includes 3-AZ HA (6 copies, <30s failover) that would otherwise require RDS Multi-AZ (~$979). Storage scales elastically — you only pay for actual data stored, no pre-allocation needed. However, the key question is: does all this data belong in a relational database? We recommend decomposing by data purpose:

Data purpose decomposition — why this matters for cost:

① **Telemetry (864M msg/month, append-only time-series):** This is the largest data volume but does NOT require real-time SQL query — it only needs periodic analytics. Route via IoT Rule → Firehose → S3 (Apache Iceberg format). Query with Athena (standard SQL, serverless, pay-per-query). Cost: ~$51/month for ingestion + storage. Athena provides full SQL capability on demand — no database infrastructure needed.

② **Operational data (user profile 5K + device registry 100K):** Small dataset, needs low-latency read/write. With telemetry offloaded to S3, Aurora only serves this lightweight workload — actual data likely <50 GB initially. Aurora Serverless v2 (min 2 ACU) or even a smaller provisioned instance (db.r7g.large = $333/month) may suffice. The 1TB / db.r7g.xlarge sizing was designed assuming ALL data goes to one DB — once telemetry is separated, the DB requirement shrinks dramatically.

Cost impact of data decomposition:

• Telemetry path (S3 Iceberg + Athena): ~$51/month — full SQL query capability, serverless, no DB infra

• Operational DB (Aurora, user+device only, ~50 GB): instance ~$333-485/month + storage ~$5-12 — vs current $560 for 1TB unused capacity

• Combined: ~$384-536/month with better architecture (HA included) vs current $560 Single-AZ without HA

By separating telemetry into S3 and keeping only operational data in the database, the DB sizing requirement drops from db.r7g.xlarge (32 GiB) to potentially db.r7g.large (16 GiB) or Aurora Serverless v2 — while gaining 3-AZ durability that the current Single-AZ estimate does not provide.

Ref:

Aurora Storage — https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Overview.StorageReliability.html

Firehose to Iceberg — https://docs.aws.amazon.com/firehose/latest/dev/apache-iceberg-destination.html

gp3 Storage — https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_Storage.html

#5: ECS Fargate ($540) + Lambda ($106) = $646/month

Realtek estimation: Fargate: 12 vCPU, 24 GB, 730 hrs (account, video, admin, bridges, workers). Lambda: 30M invocations, 1 GB, 200 ms avg.

Questions:

1. **Workload division:** Both Fargate and Lambda list account/admin workloads. What is the division? Is Fargate for long-running processes (MQTT bridge, workers) and Lambda for short-lived API calls?
2. **Lambda 30M invocations:** Does this cover only user-facing APIs (5K users × 200 calls/day × 30d = 30M)? Or does telemetry (864M) also route through Lambda?
3. **12 vCPU always-on:** What services require 24/7 compute? Could any be event-driven (Lambda) to reduce cost?

Critical note: If telemetry flows through Rules → Lambda (as your diagram suggests), then Lambda invocations should be ~864M, not 30M. This would increase Lambda cost from $106 to ~$3,000+/month. Please clarify.

Summary: Revised Cost Estimate

Item	Original	Revised	Delta
#1 CloudHSM	$2,336	$400 (Option C or E)	−$1,936
#2 IoT Core (+ Rules)	$1,650	$924 (Basic Ingest)	−$726
#3 IoT Device Mgmt	$1,135	$135	−$1,000
#4 RDS → Aurora + S3	$557	~$384 (Aurora db.r7g.large $333 + S3/Firehose $51)	−$173
#5 ECS/Lambda	$646	TBD (pending workload clarification)	TBD
Total (base)	**$4,575**	**~$2,676 (incl #5 unchanged)**	**−$1,899**

Revised per device: ~$2,676 / 100K = ~$0.027/month (vs Realtek K8s $0.03/month, vs original AWS estimate $0.046/month)

Note: This estimate assumes (1) Option C or E for cert signing, (2) Basic Ingest eligible, (3) telemetry routes to S3 not RDS, (4) ECS/Lambda unchanged pending clarification. Actual cost depends on your responses to the questions above.

Part 3: Additional Questions

OTA (firmware update) assumptions

1. OTA push frequency? (e.g., monthly / quarterly)
2. Firmware binary size? (e.g., 2MB / 5MB / 10MB)
3. Full fleet push or staged rollout?

Reference: Full push (100K devices, 5MB FW) = IoT Jobs $300 + S3 egress $60 = ~$360/push

Amazon Cognito feature requirements

Realtek estimation uses Cognito Lite tier (free). Confirm which features you need:

Tier	Cost (5K MAU)	Features
Lite	$0	Basic email/password login
Essentials	$75/month	+ MFA (TOTP/SMS), Passkey
Plus	$350/month	+ Compromised cred detection, Adaptive MFA

Reconnection storm — SLA requirement

Default limit: **500 new connections/sec/account**. 100K reconnection = ~200 sec recovery.

1. Target recovery time after fleet-wide disconnection?
2. Should we request a Service Limit Increase before load test?
3. Have you implemented exponential backoff + jitter?

Support plan inclusion

Include Business Support in unit cost? (+~$0.01/device/month)

NAT Gateway ($161/month — 2,000 GB)

- Your estimate includes 2,000 GB/month of NAT traffic. However, all AWS services in your architecture have VPC Endpoint support: S3 (Gateway, $0), ECR (Interface), IoT Core (Interface), KMS (Interface), CloudWatch Logs (Interface), SQS (Interface). With VPC Endpoints, traffic stays within AWS network — no NAT required. Estimated savings: NAT $161/month → VPC Endpoints ~$40-50/month (−$111/month).
- Question: Does any component in your architecture require outbound internet access to third-party (non-AWS) APIs? If not, NAT Gateway may be fully replaceable by VPC Endpoints.
  Ref: VPC Endpoints — https://docs.aws.amazon.com/vpc/latest/privatelink/ | ECR VPC Endpoint — https://docs.aws.amazon.com/AmazonECR/latest/userguide/vpc-endpoints.html

Managed Prometheus ($70/month) vs CloudWatch ($48/month)

- What is being monitored by each? Any overlapping metrics?
- Infrastructure metrics (ECS/Lambda/RDS)? Or device-level (100K device health)?

Data retention policy

For the data decomposition we discussed in #4 (telemetry → S3, operational → Aurora), storage cost scales with retention period. How long must each type of data be retained? (e.g., telemetry: 90 days / 1 year / 3 years? Device history: indefinite?). This determines S3 lifecycle policy (Standard → IA → Glacier) and Aurora storage growth projections.

Analytics / reporting requirements

What type of analytics do you need on telemetry/device data? Real-time dashboard (seconds)? Periodic reports (daily/weekly)? Ad-hoc SQL queries? This helps determine whether Athena on S3 (serverless, pay-per-query) is sufficient, or if a persistent analytics engine (Redshift, Managed Grafana) is needed.

Part 4: EKS 100K Load Test — Sizing Review

Approach:

1. **Establish per-pod capacity baseline —** Confirm per-pod resource limits for each critical service. Establish per-pod concurrent connection capacity.
2. **Scale incrementally —** Start small, increase load, let HPA/Cluster Autoscaler scale out. Observe saturation thresholds, extrapolate to 100K.
3. **Ensure observability —** Clear visibility into CPU/MEM/connections/queue depth at each layer. Consider CloudWatch Container Insights alongside your Prometheus + Grafana.

Goal: Derive per-node capacity ceiling + bottleneck location, then calculate 100K sizing and cost. Li-Chia will coordinate details with your team.

Questions need to be addressed:

1. **Is the resource separation appropriate? (Postgres and HAProxy deployed separately)**
2. **Is the sizing reasonable for the 100K-device load test?**
3. **Cost estimate review (EC2, EBS gp3, Postgres PVC)**

Q1. Is the resource separation appropriate? (Postgres and HAProxy deployed separately)

- **Postgres:** I understand that, due to cost considerations, RDS or other AWS managed database options are not currently preferred. However, taking database operations into account — including version upgrades, HA, backups, and operator management — if you don't already have a familiar commercial "PostgreSQL in Kubernetes" solution (e.g., Crunchy Postgres), I'd recommend pulling Postgres out of the EKS cluster and using an AWS managed database such as RDS or Aurora PostgreSQL.

  **Note: In our past experience, customers who self-operate containerized PostgreSQL clusters tend to accumulate a fair amount of technical debt — complex Operator + CRD setups increase operational complexity. That said, if you already have a PostgreSQL-on-Kubernetes solution you're comfortable with and are cost-sensitive, keeping Postgres in EKS remains a viable option.

  Reference (RDS vs. Aurora): https://aws.amazon.com/compare/aurora-and-rds/

- **HAProxy:** Separating HAProxy is a reasonable approach. However, it's worth noting that when you move to production later, you'll need to consider ingress HA. On AWS, HAProxy cannot use a VIP-floating failover mechanism like in on-premises environments — the VPC network does not allow L2 broadcast/multicast (so VRRP packets won't propagate), and each private IP is bound to a specific ENI and cannot be freely taken over by another instance.

  The recommended approach is to use an NLB for Layer 4 forwarding, with an NGINX Ingress Controller behind it to handle HTTPS, and a dedicated NLB path for MQTT. That said, if this test is purely a compute-resource load test for right-sizing — and you'd rather avoid AWS managed LB features (such as auto-scaling) affecting your test results — using a single HAProxy is still acceptable.

Q2. Is the sizing reasonable for the 100K-device load test?

- **Postgres:** If it stays within the EKS cluster, I'd recommend a memory-optimized instance such as the R-series (vCPU:RAM = 1:8), or at minimum an M-series instance like M7i (vCPU:RAM = 1:4).
- **MQTT Broker:** To handle 100K concurrent connections — each maintaining session state — I'd recommend a dedicated node with a memory-optimized or balanced instance (R- or M-series).
- **HAProxy:** A single c7i.large (2 vCPU / 4 GiB) handling 100K total connections is likely undersized. I'd recommend at least c7i.xlarge or c7i.2xlarge.

Q3. Cost estimate review (EC2, EBS gp3, Postgres PVC)

- The current cost estimate appears to be based on Graviton (ARM-based) instances, and the specs/quantities don't quite match the description in your 6/24 email. Once we've finalized the instance selection for the architecture, we can put together an updated version using the AWS Pricing Calculator.